A Generalized Software Workflow for Unanchored Approximate MUB Optimization: A Case Study in Dimension Six

Abdul Fatah, Ian McLoughlin, and Saim Ghafoor

Atlantic Technological University, Ireland

We present a reproducible, parameter-driven software workflow for optimizing approximate mutually unbiased basis configurations across dimensions, candidate basis counts, random seeds, numerical precisions, and hardware backends. The workflow uses a Lie-algebra parameterization of unitary bases, $U_k = \exp(iH_k)$, and records optimized bases, pairwise overlap tensors, optimization histories, metadata, and diagnostic summaries. The implementation was developed on consumer-level Apple Silicon hardware and designed for portable execution across CPU, Apple MPS, CUDA-capable GPU, and HPC environments. A Taylor-series matrix-exponential compatibility layer supports accelerator execution when native complex matrix exponentiation is unavailable or unsuitable, while the main structural experiments use CPU/native `matrix_exp` as the reference pathway.

As a case study, we apply the workflow to dimension six, where the existence of a complete set of seven mutually unbiased bases remains unresolved. We perform a single-seed validation sweep over $n = 2, \ldots, 7$ and 100-seed complex128/complex64 campaigns over $n = 3, \ldots, 6$. The complex128 campaign recovers exact three-basis configurations, repeatedly identifies a structured four-basis partial-exact basin with three near-exact hub edges and a defective triangle, and finds no near-exact pairs in any observed $n = 5$ or $n = 6$ run under the primary tolerance. The complex64 campaign preserves this qualitative transition for $n \geq 5$, although near-exact classifications and basin selection are more precision-sensitive. Hardware benchmarks show that accelerator execution is slower at small dimensions but becomes advantageous at larger dimensions. The results are reproducible numerical evidence about the sampled optimization landscape, not a proof of existence or nonexistence of MUB configurations in dimension six.

Abdul Fatah: abdul.fatah@research.atu.ie
Ian McLoughlin: ian.mcloughlin@atu.ie
Saim Ghafoor: saim.ghafoor@atu.ie

1 Introduction

Mutually unbiased bases (MUBs) are central objects in quantum information theory, combinatorial design, and finite-dimensional Hilbert space geometry. Two orthonormal bases B and C in $\mathbb{C}^d$ are mutually unbiased [1] if

$$|\langle b, c\rangle|^2 = \frac{1}{d} \quad \text{for all } b \in B,\ c \in C.$$

A collection of bases is mutually unbiased if every pair of bases in the collection satisfies this condition. Such structures play a fundamental role in quantum state tomography, quantum cryptography [2], and operator theory.

In dimensions that are prime or powers of primes, complete sets of $d+1$ mutually unbiased bases are known to exist [3]. In composite dimensions [4], and in particular in $d = 6$, the existence of a complete set of seven MUBs remains a long-standing open problem [5, 6]. While numerous analytic and computational approaches have been explored [7], no definitive resolution is known.

Computational searches in dimension six often reduce the search space by fixing one or more bases to canonical representatives, such as the identity, Fourier-type matrices, or prescribed

families of complex Hadamard matrices. Fixing a single basis can be interpreted as a gauge choice under the common left-unitary action and is not, by itself, necessarily restrictive. However, searches that impose additional canonical forms or restrict subsequent bases to particular Hadamard families can bias the explored landscape toward selected representatives. This motivates complementary unanchored numerical approaches in which all candidate bases are optimized simultaneously and the resulting pairwise defect geometry is analysed after optimization.

This paper adopts a mathematical-software and computational-physics perspective. We present a generalized, parameter-driven software workflow for optimizing and analysing approximate mutually unbiased basis (AMUB) configurations across dimensions, candidate basis counts, random seeds, numerical precisions, and hardware backends. The aim is not to provide a proof of existence or nonexistence for complete MUB sets in dimension six, but to provide a reproducible computational framework for exploring AMUB landscapes and comparing the structures reached by a specified optimizer, parameterization, precision policy, and seed set.

The workflow uses an unanchored formulation in which all candidate bases are treated symmetrically during optimization. Each candidate basis is parameterized as a unitary matrix through Lie-algebra exponentiation [8],

$$U_k = \exp(iH_k), \qquad H_k = H_k^\dagger,$$

where the Hermitian generator H_k is obtained from an unconstrained trainable complex matrix. This construction enforces the unitary constraint through the forward model rather than through projection steps or penalty terms. The AMUB objective measures the total pairwise deviation from the mutually unbiased condition, while retaining the individual pairwise defects as diagnostic quantities. This pairwise decomposition allows each optimized configuration to be interpreted as a weighted complete graph whose edge weights quantify residual AMUB defect.

The implementation is written in Python using PyTorch and NumPy, with configuration-driven execution, structured artifact export, and reproducible post-processing. Each run records optimized basis matrices, pairwise overlap tensors, pairwise diagnostics, unitarity residuals, generator norms, optimization histories, and run metadata. The main structural and precision-comparison experiments use the CPU pathway with native `torch.matrix_exp` as the reference implementation. For accelerator execution, the workflow includes a Taylor-series matrix-exponential compatibility layer, used when native complex matrix exponentiation is unavailable or unsuitable on a given backend. This design separates the reference numerical experiments from backend-specific acceleration and benchmarking.

As a case study, we apply the workflow to the dimension-six problem. We first perform a single-seed validation sweep over $n = 2, \ldots, 7$, ranging from the positive-control two-basis case to the formal complete-set target $n = d + 1 = 7$. We then perform 100-seed campaigns for $n = 3, 4, 5, 6$, which form the main nontrivial transition regime studied in this work. These campaigns allow us to distinguish recurrent structural features from initialization-dependent local basins.

The resulting dimension-six experiments show a structured progression. For $n = 3$, the workflow recovers exact three-basis configurations, while also revealing defective local basins for some seeds. For $n = 4$, the workflow repeatedly identifies a partial-exact hub-and-triangle structure: three basis pairs are near-exact, while the remaining three pairs form a localized defective triangle. Thus, the observed four-basis configurations are not exact four-MUB configurations, but structured partial-exact configurations. For $n = 5$ and $n = 6$, no near-exact pairs are observed in the 100-seed campaigns under the primary tolerance, indicating fully defective configurations within the sampled optimization landscape.

The paper also investigates numerical precision by comparing double-precision `complex128` reference campaigns with reduced-precision `complex64` campaigns. Both precisions use the same CPU/native-matrix-exponential structural pathway, allowing precision effects to be separated from accelerator-specific Taylor-exponential effects. The comparison focuses on loss values, near-exact pair counts, tolerance sensitivity, basin selection, and unitarity residuals. The qualitative transition to fully defective observed configurations for $n \geq 5$ is stable across precisions, while near-exact classifications and basin selection are more sensitive in reduced

precision.

Finally, we benchmark the workflow across increasing dimensions and hardware backends. These benchmarks are intended to assess portability and scaling rather than to define the reference structural results. They show that accelerator execution is slower for small dimensions, where fixed launch overheads dominate, but becomes advantageous at larger dimensions as dense linear-algebra workloads increase. In this way, the software supports development on consumer-level hardware, reference CPU campaigns, and scalable execution on HPC resources.

Overall, this work contributes a reproducible computational framework for AMUB optimization, together with a dimension-six case study that exposes exact triples, structured four-basis partial-exact basins, and fully defective observed configurations for $n = 5$ and $n = 6$ under the stated workflow and tolerance policy. The results should be interpreted as reproducible numerical evidence about the optimization landscape sampled by this software, not as a proof of existence or nonexistence of complete MUB configurations in dimension six.

2 Contributions

This paper makes the following contributions.

Reproducible AMUB software workflow. We design and implement a parameter-driven software workflow for optimizing approximate mutually unbiased basis configurations across dimensions, candidate basis counts, random seeds, numerical precisions, and hardware backends. The workflow exports self-contained run artifacts, including optimized basis matrices, pairwise overlap tensors, pairwise diagnostics, unitarity residuals, generator norms, optimization histories, and run metadata. This artifact-based design separates optimization from post-processing and supports independent inspection, regeneration, and extension of the reported experiments.

Unanchored Lie-algebra optimization model. We formulate the AMUB search as an unanchored optimization problem in which all candidate bases are optimized simultaneously. Each basis is parameterized by Lie-algebra exponentiation,

$$U_k = \exp(iH_k), \qquad H_k = H_k^\dagger,$$

with Hermitian generators obtained from unconstrained trainable complex matrices. This enforces unitarity through the forward model rather than through projection steps or penalty terms, while retaining the symmetry of the unanchored formulation during optimization.

Pairwise defect diagnostics. Beyond recording the aggregate AMUB loss, the workflow retains individual pairwise defects, maximum entrywise deviations from $1/d$, near-exact classifications under multiple tolerances, and overlap summary statistics. This allows each optimized configuration to be analysed as a weighted complete graph whose edge weights quantify pairwise AMUB defect, making it possible to distinguish exact triples, partial-exact hub structures, localized defective subgraphs, and fully defective observed configurations.

Backend-aware matrix-exponential policy. The main structural and precision-comparison experiments use the CPU/native-`matrix_exp` pathway as the reference implementation. To support accelerator execution when native complex matrix exponentiation is unavailable or unsuitable, the workflow also includes a Taylor-series matrix-exponential compatibility layer. This layer is used for accelerator execution and hardware benchmarking, while the structural AMUB conclusions are based on the CPU reference pathway.

Dimension-six validation and multi-seed case study. We apply the workflow to dimension six using a single-seed validation sweep over $n = 2, \ldots, 7$ and 100-seed campaigns over $n = 3, 4, 5, 6$. The single-seed sweep verifies that the same implementation runs from the positive-control two-basis case to the formal complete-set target $n = d + 1 = 7$. The 100-seed campaigns provide a reproducible empirical sample of the nonconvex optimization landscape for the main transition regime.

Numerical evidence for structured defect in $d = 6$. In the complex128 reference campaign, the workflow recovers exact three-basis configurations and repeatedly identifies a structured four-basis partial-exact basin with three near-exact pairs and three defective pairs. For $n = 5$ and $n = 6$, no near-exact pairs are observed in any of the 100-seed complex128 or complex64 campaigns under the primary tolerance. These results provide reproducible numerical evidence about the

configurations sampled by the present workflow, not a proof of existence or nonexistence of complete MUB sets in dimension six.

Precision-aware analysis. We compare complex128 and complex64 campaigns using the same CPU/native-matrix-exponential structural pathway. The comparison shows that the qualitative disappearance of near-exact pairs for $n = 5$ and $n = 6$ is stable across the two precisions, while near-exact classifications and basin selection, especially for $n = 4$, are more sensitive in complex64 arithmetic.

Hardware portability and scaling benchmarks. We benchmark the workflow across increasing dimensions and hardware backends. On the tested benchmark grid $d \in \{6, 12, 24, 48, 96\}$, GPU execution is slower at small dimensions but becomes advantageous at larger dimensions: the crossover occurs between $d = 24$ and $d = 48$ for complex128 and between $d = 48$ and $d = 96$ for complex64. These benchmarks demonstrate the portability of the same software stack across consumer and HPC-style execution environments.

Reproducible visualization and representative-run selection. The workflow produces aggregate summaries and representative-run manifests for visualization. These include representative positive controls, exact or near-exact triples, structured four-basis partial-exact configurations, fully defective observed $n = 5$ and $n = 6$ configurations, and an exploratory $n = 7$ complete-target run. The visualization stage reads saved artifacts rather than rerunning optimization, supporting reproducible figure generation.

3 Mathematical Formulation

This section defines the approximate mutually unbiased basis problem studied in this paper. The main numerical case study is carried out in dimension $d = 6$. We therefore consider collections of n candidate orthonormal bases in the complex Hilbert space $\mathbb{C}^6$ [9]. Each basis is represented by a unitary matrix [10] $U_k \in U(6)$, whose columns are the basis vectors. The formulation below is written explicitly for $d = 6$, while the software implementation is parameterized by the dimension.

3.1 Mutual Unbiasedness

Let $B_i = \{u_{i,1}, \ldots, u_{i,6}\}$ and $B_j = \{u_{j,1}, \ldots, u_{j,6}\}$ be two orthonormal bases of $\mathbb{C}^6$. The bases are mutually unbiased [3] if every vector in one basis has equal squared overlap with every vector in the other basis:

$$|\langle u_{i,a}, u_{j,b}\rangle|^2 = \frac{1}{6}, \qquad 1 \le a, b \le 6. \tag{1}$$

Thus, measurement of a state prepared in one basis gives a uniform probability distribution when expressed in the other basis.

Equivalently, if the bases are represented by unitary matrices U_i and U_j, with columns given by the corresponding basis vectors, then the cross-Gram matrix $U_i^\dagger U_j$ satisfies

$$|(U_i^\dagger U_j)_{ab}|^2 = \frac{1}{6}, \qquad 1 \le a, b \le 6, \tag{2}$$

In other words, all entries of $U_i^\dagger U_j$ have magnitude $1/\sqrt{6}$. Although $U_i^\dagger U_j$ is itself unitary, the mutual unbiasedness condition constrains only its entrywise squared moduli. In matrix form, the condition is

$$|U_i^\dagger U_j|^2 = \frac{1}{6}\mathbf{1}_{6\times 6}, \tag{3}$$

where the absolute value and square are taken entrywise, and $\mathbf{1}_{6\times 6}$ denotes the 6×6 all-ones matrix.

For an exact collection of n mutually unbiased bases, condition (3) must hold for every pair $1 \le i < j \le n$. In this paper, we do not attempt to certify exact existence or nonexistence of complete MUB sets in dimension six. Instead, we use a continuous optimization loss function to study approximate configurations and the pairwise defect structures of low-loss configurations found by the specified numerical workflow.

3.2 Approximate MUB Loss

For a pair of candidate bases U_i and U_j, define the squared-modulus overlap matrix

$$M_{ij} = |U_i^\dagger U_j|^2,$$

where the square is taken entrywise. The pairwise approximate-MUB defect is then defined as the squared Frobenius deviation of M_{ij} from the uniform target matrix:

$$\ell_{ij} = \left\| M_{ij} - \frac{1}{6}\mathbf{1}_{6\times 6} \right\|_F^2 = \left\| |U_i^\dagger U_j|^2 - \frac{1}{6}\mathbf{1}_{6\times 6} \right\|_F^2. \tag{4}$$

Thus, $\ell_{ij} = 0$ if and only if the pair (U_i, U_j) satisfies the mutual unbiasedness condition exactly.

The total n-basis loss is the sum of all pairwise defects:

$$\mathcal{L}_n = \sum_{1\le i<j\le n} \ell_{ij} = \sum_{1\le i<j\le n} \left\| |U_i^\dagger U_j|^2 - \frac{1}{6}\mathbf{1}_{6\times 6} \right\|_F^2 . \tag{5}$$

Consequently, $\mathcal{L}_n = 0$ corresponds to an exact mutually unbiased collection of n bases. In numerical experiments, nonzero values of $\mathcal{L}_n$ quantify the aggregate residual deviation from mutual unbiasedness.

The loss admits a natural pairwise decomposition, which is important for the analysis in this work. Rather than recording only the scalar value $\mathcal{L}_n$, the workflow retains the individual quantities ℓ_{ij} for all unordered basis pairs. These pairwise terms reveal whether residual defect is localized in a small number of basis pairs or distributed across the entire configuration. Equivalently, an optimized configuration can be viewed as a weighted complete graph on n vertices: vertices correspond to bases, and edge weights correspond to pairwise defects ℓ_{ij}. This representation is used later to distinguish exact pairs, partial-exact structures, and fully defective observed configurations.

3.3 Unanchored Optimization

The optimization problem studied here is unanchored, no basis is fixed in advance to the identity, to a Fourier basis, or to any other canonical representative. Instead, all candidate bases $U_1, \dots, U_n$ are optimized simultaneously. The unanchored AMUB optimization problem is therefore

$$\min_{U_1,\dots,U_n \in U(6)} \mathcal{L}_n(U_1, \dots, U_n). \tag{6}$$

This formulation treats all candidate bases symmetrically during optimization. The objective is invariant under a common left unitary action,

$$(U_1, \dots, U_n) \mapsto (WU_1, \dots, WU_n), \qquad W \in U(6),$$

because

$$(WU_i)^\dagger (WU_j) = U_i^\dagger U_j.$$

Thus, the unanchored formulation retains a global unitary gauge freedom.

Anchored formulations are often used for classification and for fixed unitary-equivalence freedoms [11, 12], Fixing a single basis, for example $U_1 = I$, can be interpreted as a gauge choice under the common left-unitary symmetry. Such a gauge fixing is not, by itself, necessarily a restriction of the underlying equivalence class. However, practical anchored searches may impose additional structure, such as fixing further bases to prescribed Fourier or Hadamard-family representatives. Those additional choices can restrict the explored numerical landscape.

The unanchored formulation used here avoids imposing such representatives during optimization. All bases are allowed to move simultaneously, and structural features are extracted only after optimization from the recorded pairwise defects. This is particularly useful for the present numerical study, where empirically observed configurations can be analysed through their pairwise defect geometry. In later sections, this viewpoint is used to describe structures such as exact triples, hub-like partial-exact configurations, and fully defective observed configurations. These empirical structures are discussed in the context of known rigidity and nonextendability phenomena for MUB configurations in dimension six [13].

3.4 Lie-Algebra Parameterization of Unitary Bases

To enforce unitarity by construction, each candidate basis is parameterized using Lie-algebra exponentiation [8]. For each basis index k, we define

$$U_k = \exp(iH_k), \qquad H_k = H_k^\dagger. \tag{7}$$

Since H_k is Hermitian, iH_k is skew-Hermitian, and therefore $\exp(iH_k)$ is unitary in exact arithmetic. This uses the Lie group–Lie algebra relation for the unitary group. The representation is not unique, but it provides a differentiable parameterization that is convenient for unconstrained numerical optimization.

In the implementation, H_k is obtained from an unconstrained trainable complex matrix $A_k \in \mathbb{C}^{d\times d}$ by Hermitian symmetrization:

$$H_k = \frac{A_k + A_k^\dagger}{2}. \tag{8}$$

This construction discards the anti-Hermitian component of A_k, ensuring that the generator

used in the exponential is Hermitian. The resulting representation is redundant: the complex matrix A_k contains $2d^2$ real degrees of freedom, whereas the space of Hermitian $d \times d$ matrices has real dimension d^2. This redundancy is accepted deliberately because it allows the trainable variables to remain unconstrained complex matrices while the forward model enforces the Hermitian and unitary structure.

The trainable variables are therefore the unconstrained matrices $A_1, \ldots, A_n$, while the matrices appearing in the AMUB loss are the unitary matrices generated by (7). This separates the unitary constraint from the AMUB objective: the optimizer does not need to enforce orthonormality through projection steps or penalty terms. Instead, deviations from exact unitarity arise only from finite-precision arithmetic and the numerical matrix-exponential pathway, and are monitored separately through the unitarity residuals defined below.

Combining (7) and (8), the computational parameterization is

$$U_k(A_k) = \exp\left(i\frac{A_k + A_k^\dagger}{2}\right), \qquad k = 1, \ldots, n. \tag{9}$$

The map $A_k \mapsto U_k(A_k)$ is differentiable through the operations used in the forward model, so gradients of the AMUB loss can be propagated through the matrix exponential using automatic differentiation [14].

For the dimension-six case study, the resulting numerical optimization problem can then be written as

$$\min_{A_1,\ldots,A_n \in \mathbb{C}^{6\times 6}} \mathcal{L}_n\left(U_1(A_1), \ldots, U_n(A_n)\right). \tag{10}$$

Although the optimization variables in (10) are unconstrained matrices, the corresponding bases used in the loss are unitary by construction in exact arithmetic.

3.5 Diagnostics Derived from the Loss

The total loss $\mathcal{L}_n$ provides a scalar measure of aggregate approximate MUB defect, but it does not by itself describe how the residual error is distributed across the basis pairs. For this reason, the workflow records pairwise diagnostics in addition to the aggregate objective value.

For each unanchored pair (i, j), we compute the squared-modulus overlap matrix

$$M_{ij} = |U_i^\dagger U_j|^2, \tag{11}$$

where the absolute value and square are taken entrywise. The corresponding pairwise loss is

$$\ell_{ij} = \left\| M_{ij} - \frac{1}{6}\mathbf{1}_{6\times 6} \right\|_F^2, \tag{12}$$

which is the same pairwise defect used in the total loss (5). We also record the maximum entrywise deviation from the target overlap value,

$$\delta_{ij} = \max_{1\le a,b\le 6} \left| (M_{ij})_{ab} - \frac{1}{6} \right|. \tag{13}$$

The quantity ℓ_{ij} measures the total squared deviation of a pair, while δ_{ij} records the worst entrywise deviation. These two diagnostics are complementary: two pairs may have similar Frobenius defects but different maximum entrywise deviations.

Given a tolerance $\tau > 0$, a pair (i, j) is classified as near-exact if

$$\delta_{ij} \le \tau. \tag{14}$$

The primary tolerance used in the reported summaries is $\tau = 10^{-6}$. For precision-sensitivity analysis, especially in reduced precision, near-exact counts are also recomputed using relaxed tolerances such as 10^{-5} and 10^{-4}. This separates the recorded numerical quantities ℓ_{ij} and δ_{ij} from the tolerance-dependent classification rule.

In addition to pairwise AMUB diagnostics, we monitor numerical unitarity by computing, for each optimized basis,

$$\epsilon_k = \|U_k^\dagger U_k - I_6\|_F. \tag{15}$$

Although the Lie-exponential parameterization enforces unitarity in exact arithmetic, ϵ_k measures deviations introduced by finite-precision arithmetic and numerical matrix exponentiation. These residuals are used as a consistency check: observed AMUB defects should not be attributed to loss of orthonormality unless the corresponding unitarity residuals are large enough to explain them.

The workflow also records generator-norm diagnostics for the Hermitian matrices H_k. These diagnostics are used to monitor the scale of the Lie-algebra parameters and, when the Taylor-series matrix-exponential compatibility layer is

used for accelerator execution, to evaluate the relevant truncation-error bounds under the observed generator norms.

3.6 Interpretation as Pairwise Defect Geometry

The pairwise decomposition of $\mathcal{L}_n$ is central to the interpretation of the numerical results. Each optimized configuration determines a weighted complete graph with vertex set $\{1, \ldots, n\}$. The vertices represent candidate bases, and the edge weight between vertices i and j is the pairwise AMUB defect ℓ_{ij}. We refer to this weighted-graph representation as the pairwise defect geometry of the configuration.

In this representation, near-zero edges correspond to exact or near-exact MUB pairs under the chosen tolerance, while nonzero edges indicate defective pairwise relations. This graph viewpoint allows configurations with the same or similar aggregate loss to be distinguished by how their residual defect is distributed. For example, a configuration may contain one basis that forms near-exact pairs with several others, together with a localized defective triangle among the remaining bases. Alternatively, it may exhibit fully distributed defect, with every pair carrying nonzero loss.

This interpretation is particularly useful in the dimension-six experiments reported later. It allows the results to be described not only by scalar loss values, but also by structural patterns in the pairwise defects. In particular, the pairwise defect geometry is used to compare exact three-basis configurations, structured four-basis partial-exact configurations, and fully defective observed configurations for larger candidate basis counts. The same diagnostics also support comparison across numerical precisions, since near-exact classifications may depend on tolerance while the underlying pairwise losses and maximum deviations remain explicitly recorded.

4 Software Workflow and Precision Policy

This section describes the computational workflow used to generate and analyze the approximate MUB configurations. The workflow is designed as a reproducible mathematical-software artifact rather than as a one-off numerical experiment. It records the optimization configuration, numerical precision, backend policy, basis matrices, overlap matrices, diagnostics, and summary tables in machine-readable formats. The experimental protocol built on this workflow is described in Section 5, and the resulting numerical evidence is reported in Section 6.

4.1 Implementation Overview

The implementation is written in Python and uses PyTorch for differentiable optimization, NumPy for array storage and post-processing, and marimo notebooks for executable computational narratives. The notebook interface is used for executable presentation, while the core experiment functions are parameterized and reusable outside the notebook environment. The workflow is organized around a single parameterized experiment function that accepts the dimension d, the number of candidate bases n, the numerical dtype, random seed, optimizer settings, and output directory.

For each experiment, the workflow performs the following steps:

1. construct an unanchored Lie-exponential AMUB model with n trainable bases in $\mathbb{C}^d$;
2. optimize the total pairwise AMUB loss using Adam;
3. record the best basis configuration encountered during optimization;
4. compute pairwise overlap diagnostics and unitarity residuals;
5. save basis matrices, overlap matrices, diagnostics, optimization histories, and run metadata;
6. aggregate results across seeds and precisions for later analysis.

This design allows the same code path to be used for single-run validation, multi-seed campaigns, and precision-comparison experiments. The implementation supports the full range $n = 2, \ldots, 7$ used in the exploratory sweep, while the multi-seed campaigns focus on $n = 3, \ldots, 6$.

4.2 Backend Policy and Hardware Acceleration via Taylor Series

The local reference platform used for the experiments is an Apple Silicon system. PyTorch [15] detected the Metal Performance Shaders (MPS) backend, which provides access to Apple GPU execution through the `mps` device. However, the Lie-algebra parameterization used in this workflow requires repeated evaluation of the complex matrix exponential $\exp(iH_k)$. In the local PyTorch/MPS environment, the required complex `torch.matrix_exp` operation was not natively implemented on the MPS backend. To resolve this limitation and support portable execution on non-CUDA accelerator backends, we developed a customized Taylor-series matrix exponentiation layer:

$$T_N(X) = \sum_{m=0}^{N} \frac{X^m}{m!}, \qquad \exp(X) \approx T_N(X) \quad (16)$$

To quantify the truncation error of this approximation, we use the following spectral-norm bound. For a skew-Hermitian matrix $X = iH$, with $H = H^\dagger$, the truncation error of the order-N Taylor series $T_N(X)$ in the spectral norm is bounded by:

$$\| \exp(X) - T_N(X) \|_2 \leq \frac{\|X\|_2^{N+1}}{(N+1)!} e^{\|X\|_2} \quad (17)$$

For $X = iH$ with $H = H^\dagger$, we have $\|X\|_2 = \|H\|_2$. Although every unitary matrix admits a principal Hermitian logarithm with spectral norm at most π, the trainable generators in our parameterization are not explicitly constrained to this principal branch. Therefore, the workflow records generator spectral norms at saved checkpoints and at the best saved iterate, evaluating the truncation bound using the observed norms. This is the standard Taylor remainder estimate for the matrix exponential in a submultiplicative norm [16].

Across the reported multi-seed campaigns and precisions, the maximum generator norm observed at the best saved iterate was approximately $\|H_k\|_2 = 2.71$. With $N = 20$, the truncation bound evaluated at this observed maximum norm is

$$\frac{2.71^{21}}{21!} e^{2.71} \approx 3.64 \times 10^{-10}.$$

To guarantee precision bounds in generalized, higher-dimensional searches, the software dynamically monitors the Frobenius norm of the generator matrix. If the norm exceeds a threshold of 3.0, the workflow issues a warning suggesting that the Taylor order N be increased or the optimizer parameters adjusted. This bound concerns the mathematical Taylor truncation error only. Floating-point accumulation, matrix-multiplication roundoff, and backend-specific numerical effects are monitored through the recorded unitarity residuals and precision-comparison diagnostics. Because $T_N(iH)$ is not exactly unitary in finite-precision arithmetic, Taylor-based GPU runs are interpreted as approximate exponential runs rather than exact Lie-exponential evaluations, and they are accompanied by the unitarity residual diagnostic $\|U_k^\dagger U_k - I_d\|_F$ defined in Section 3. Empirically, these monitored residuals remain extremely small, consistently measuring below 10^{-9} for both the Apple Silicon MPS and NVIDIA A100 CUDA runs, confirming that the Taylor series approximation maintains numerical unitarity below the physical tolerance thresholds of the MUB search. Since this custom layer relies exclusively on dense matrix multiplication and addition, it bypasses the MPS complex matrix exponential limitation and enables execution on the Apple M1 GPU without CPU fallback.

In our workflow, the implementation automatically routes matrix exponentials based on the active device:

- On CPU, it uses PyTorch's native `torch.matrix_exp(1j * H)` as reference implementation.
- On GPU (`mps` or `cuda`), it dynamically employs the Taylor-series expansion layer to avoid `NotImplementedError` or CPU-fallback bottlenecks.

This backend policy enables running the identical unanchored AMUB optimization workflow across CPUs, Apple Silicon GPUs, and NVIDIA CUDA platforms, which we evaluate in Section 6.6.

4.3 Precision Policy

The workflow treats numerical precision as part of the experimental configuration. The complex128

implementation is used as the reference precision because the diagnostics involve small residuals, near-exact pair classifications, and unitarity checks. The workflow records sufficient diagnostics to test the stability of near-exact classifications under multiple tolerances.

A complex64 version of the same workflow is used as a precision-sensitivity experiment. The purpose of the complex64 campaign is not to replace the complex128 reference, but to examine whether the qualitative structure of the AMUB landscape is preserved under reduced complex precision. In particular, the workflow compares loss values, near-exact pair counts, basin diversity, and unitarity residuals across complex128 and complex64.

For near-exact pair classification, the primary tolerance is 10^{-6} in the reported summaries. Because complex64 arithmetic has lower numerical resolution, the workflow also recomputes near-exact counts using relaxed tolerances, including 10^{-5} and 10^{-4}, to distinguish genuine structural changes from tolerance-sensitive classifications.

4.4 Optimization Components

Each run constructs a model consisting of n trainable complex matrices $A_1, \ldots, A_n$. These matrices are converted to Hermitian generators by

$$H_k = \frac{A_k + A_k^\dagger}{2},$$

and each basis is generated by

$$U_k = \exp(iH_k).$$

The AMUB loss is then evaluated over all $\binom{n}{2}$ basis pairs.

The optimization uses the Adam optimizer with fixed hyperparameters recorded in the run metadata and summarized for complete reproducibility in Table 1 (see Section 5). During optimization, the workflow tracks both the current loss and the best loss encountered. The basis matrices corresponding to the best observed loss are retained for post-processing and artifact export.

The workflow does not claim that the resulting configurations are global minimizers. The objective is nonconvex, and different random seeds may converge to different basins. For this reason, multi-seed campaigns are used to identify recurrent structures and basin diversity.

4.5 Saved Artifacts

For each run, the workflow writes a self-contained artifact directory. The directory name encodes the dimension, number of bases, dtype, and seed. Each run directory contains:

- `best_bases.npy`, storing the optimized basis matrices;
- `pairwise_overlaps.npz`, storing all matrices $|U_i^\dagger U_j|^2$;
- `pairwise_diagnostics.json`, storing pairwise losses, maximum deviations, and overlap statistics;
- `unitarity_residuals.json`, storing the residuals $\|U_i^\dagger U_i - I\|_2$ to verify numerical orthonormality;
- `generator_norms.json`, storing norms of the Lie-algebra generator parameters;
- `optimization_history.csv`, storing the loss trace recorded during optimization;
- `run_metadata.json`, storing problem parameters, optimizer settings, dtype, backend policy, and diagnostic summaries.

Campaign-level summaries are also saved. These include per-run CSV files, aggregate JSON summaries by n and dtype, precision-comparison summaries, representative-run manifests, and figure-data JSON files. The artifact structure is intended to make it possible to inspect the reported results without rerunning the full optimization.

4.6 Visualization Workflow

The visualization stage is separated from the optimization stage. Rather than recomputing optimized bases, visualization routines load the saved overlap matrices and diagnostics from disk. This separation reduces ambiguity between computation and post-processing.

The workflow produces two types of visual summaries. First, aggregate plots show loss statistics and near-exact pair counts as functions of n and numerical precision. Second, sorted pairwise-loss spectra show the distribution of pairwise defects within representative configurations. These compact spectra are used in

the main paper because full heatmap grids become large as n increases. Full pairwise overlap heatmaps are retained as supporting artifacts.

4.7 Software Generalization and Multi-Dimensional Scalability

Although the experiments in this paper focus on the longstanding open problem in $d = 6$, the PyTorch-based unanchored software architecture is fully generalized and scalable to arbitrary dimensions d and candidate basis counts n. A researcher can adapt the workflow to study approximate MUBs in $d = 7, 8$, or composite dimensions like $d = 10$ and $d = 12$ simply by passing the desired parameters to the core model instantiation:

```
model = UnanchoredAMUBModel(d=10, n_bases=8)
```

The same software architecture may be adapted to related unitary-constrained search problems, such as Quantum Latin Squares [17], Unitary Error Bases [18], and biunitary matrices [19], after replacing the AMUB loss with problem-specific constraint losses.

As the dimension increases, the dense matrix operations become more computationally intensive, and hardware acceleration is expected to become more favorable. The benchmark results in 6.6 quantify this trend for the tested dimensions and extrapolate the observed crossover behavior. The benchmark results suggest that, as dimension increases, the larger dense linear-algebra workload can amortize fixed accelerator-launch overheads, making GPU execution increasingly favorable for larger-dimensional searches such as larger-dimensional quantum combinatorial design searches.

5 Experimental Methodology

This section describes the numerical experiments used to evaluate the unanchored AMUB workflow in dimension six. The experiments are designed to study the structure of optimized approximate MUB configurations as the number of candidate bases increases, and to assess the sensitivity of the observed structures to random initialization and numerical precision.

5.1 Experimental Goals

The experiments address three questions.

First, we ask how the optimized AMUB configurations change as the number of candidate bases increases from small collections to the complete-set target size. The complete-set target $n = 7$ in $d = 6$is included as an exploratory single-seed case, while the main multi-seed analysis focuses on $n = 3, \ldots, 6$.

Second, we ask whether observed structures are robust to random initialization. Because the objective is nonconvex, a single run may not be representative of the optimization landscape. We therefore perform multi-seed campaigns for selected values of n.

Third, we ask whether the qualitative structures observed in the complex128 reference workflow persist under complex64 execution. This precision comparison is intended to distinguish robust structural features from precision-sensitive diagnostics or basin-selection effects.

5.2 Single-Seed Sweep over Candidate Basis Counts

As an initial validation and exploratory sweep, the workflow is run for $n = 2, \ldots, 7$ using the complex128 reference precision and a fixed random seed. This sweep serves two purposes. First, it verifies that the same parameterized implementation can handle all candidate-basis counts in the range $2 \leq n \leq 7$, from the basic two-basis case to the formal complete-set target $n = d+1 = 7$ in dimension $d = 6$. Second, it produces representative configurations and saved artifacts for preliminary inspection before the multi-seed campaigns.

The case $n = 2$ is included as a positive control, since an unbiased pair is known to be attainable and should correspond to a near-zero AMUB loss under successful optimization. The case $n = 7$ is included as an exploratory complete-target run, but it is not used by itself to draw statistical conclusions about the existence or nonexistence of complete MUB sets in dimension six.

For each value of n, the workflow constructs n trainable bases in $\mathbb{C}^6$, optimizes the AMUB loss, records the best configuration encountered during optimization, and saves the corresponding artifacts. The number of unordered pairwise AMUB constraints grows as

$$\binom{n}{2},$$

so the sweep also provides a basic computational check that the implementation scales consistently with the number of basis pairs.

The single-seed sweep is not used as a statistical characterization of the nonconvex optimization landscape. Instead, it serves as a validation and exploratory step that motivates the subsequent multi-seed campaigns, where initialization sensitivity, basin diversity, and precision effects are examined more systematically.

5.3 Multi-Seed Campaigns

To assess sensitivity to initialization, we perform multi-seed campaigns for $n = 3, 4, 5, 6$. These values are selected because they cover the transition from exact or partially exact configurations to fully defective configurations in the observed landscape. The case $n = 2$ is used as a basic positive-control case and is not the focus of the multi-seed study. The case $n = 7$ is included in the single-seed sweep as the complete-set target, but the multi-seed campaigns reported here focus on $n = 3, \ldots, 6$.

For each n in the multi-seed campaign, the workflow runs 100 independent seeds:

$$s \in \{0, 1, \ldots, 99\}.$$

This larger-scale campaign is designed to provide a robust characterization of basin diversity and stability across random initializations. The workflow is designed to support arbitrary seed pools without changing the experimental code path.

For every pair (n, s), the model is reinitialized, optimized, diagnosed, and saved independently. The reported multi-seed summaries include the minimum, median, and maximum best loss over the 100 seeds, and the min/median/max near-exact MUB pairs observed.

5.4 Optimization Settings

All experiments use the same unanchored Lie-exponential parameterization described in Section 3. The trainable variables are unconstrained complex matrices A_k, which are symmetrized to Hermitian matrices H_k before exponentiation.

The optimizer is Adam. All important optimization and campaign-level hyperparameters are summarized in Table 1 to ensure full experimental reproducibility. The optimization history stores the current loss and best loss at regular logging intervals.

During optimization, the workflow tracks the best loss encountered rather than relying only on the final iterate. This is important because the objective is nonconvex and the Adam iterates may fluctuate within a basin. The basis matrices corresponding to the best observed loss are retained and used for all subsequent diagnostics and artifact generation.

As detailed in Table 1, the workflow uses a base step count of 1500 steps for smaller configurations ($n < 5$) and an increased step count of 2000 steps for harder cases with $n \geq 5$ bases to allow sufficient convergence in the higher-dimensional constraint landscapes.

5.5 Precision Campaigns

The complex128 workflow is treated as the reference implementation. This precision is used for the main structural analysis because the diagnostics include small unitarity residuals, near-exact pair classifications, and pairwise deviations from 1/6.

A corresponding complex64 campaign is run using the same parameterized workflow, the same values of n, and the same seed set for the multi-seed experiments. The purpose of the complex64 campaign is not to replace the complex128 reference results, but to evaluate whether the same qualitative structures persist under reduced complex precision.

To isolate numerical precision effects from Taylor truncation errors, the primary structural AMUB experiments and precision-comparison campaigns are executed entirely on the CPU backend using PyTorch's native `torch.matrix_exp` implementation. Unless otherwise stated, the precision-comparison campaigns use this reference CPU pathway, separating backend-specific Taylor GPU execution and timing experiments (Section 6.6) from the AMUB structural analysis.

The precision comparison focuses on:

- best loss statistics over seeds;
- number of near-exact basis pairs;
- basin diversity as indicated by rounded best losses;
- unitarity residuals;

Table 1: Optimization and campaign-level hyperparameters used in the reported AMUB campaigns. The Taylor order applies only to GPU-backend timing experiments.

Hyperparameter	Value / Setting
Optimizer	Adam
Learning rate (η)	0.02
Initialization scale	0.05
Step count (for $n < 5$)	1500 steps
Step count (for $n \geq 5$)	2000 steps
Primary numerical precision	`complex128` (double-precision reference)
Secondary numerical precision	`complex64` (single-precision baseline)
Campaign seed set (s)	$\{0, 1, \ldots, 99\}$
Single-seed validation seed (s)	1234
Primary near-exact tolerance (τ)	10^{-6}
Taylor expansion order (N)	20 (utilized on GPU backends)

- stability of the transition from partial exactness to fully defective configurations.

5.6 Near-Exact Pair Classification

For each optimized configuration, we compute the maximum entrywise deviation

$$\delta_{ij} = \max_{a,b} \left| \left(|U_i^\dagger U_j|^2 \right)_{ab} - \frac{1}{6} \right|$$

for every pair (i, j). A pair is classified as near-exact $\delta_{ij} \leq \tau$.

The primary tolerance used in the summary tables is

$$10^{-6}.$$

For precision-sensitivity analysis, particularly for complex64 runs, the near-exact counts are also recomputed using relaxed tolerances

$$10^{-5} \quad \text{and} \quad 10^{-4}.$$

This additional check is used to determine whether changes in near-exact pair counts reflect genuine structural differences or tolerance-sensitive numerical effects.

5.7 Pairwise Structural Diagnostics

The scalar loss $\mathcal{L}_n$ is not sufficient to characterize the geometry of an optimized configuration. Therefore, the workflow records pairwise diagnostics for every pair (i, j), including:

- pairwise loss contribution ℓ_{ij};
- minimum, maximum, mean, and standard deviation of $|U_i^\dagger U_j|^2$;
- maximum absolute deviation δ_{ij} from $1/6$;
- near-exact classification under the selected tolerance.

The pairwise losses are used to identify structural patterns such as defective triangles, hub-and-triangle configurations, and globally distributed defect. Here a hub-and-triangle configuration denotes one basis forming near-exact pairs with several others, while the remaining bases form a localized defective triangle. For visualization, the pairwise losses are sorted from largest to smallest, producing a compact spectrum of pairwise defects for each representative configuration.

5.8 Representative Configurations

After the multi-seed campaigns, representative runs are selected for visualization. The representative selection is based on the saved run summaries and diagnostics rather than manual inspection of heatmaps. Within each structural category, the lowest-loss run is selected as the representative. If multiple runs have the same rounded loss, the smallest seed index is used as a deterministic tie-breaker.

The representative set includes, when present:

- an exact or near-exact $n = 3$ configuration;
- a defective $n = 3$ triangular configuration;
- an $n = 4$ hub-and-triangle configuration;
- a best observed fully defective $n = 5$ configuration;

- a best observed $n = 6$ low-loss configuration;
- a recurrent $n = 6$ basin when available.

These representatives are used to generate sorted pairwise-loss spectra and supporting heatmap visualizations. The representative-run manifest records the selected run directory, seed, dtype, best loss, number of near-exact pairs, and paths to the saved diagnostics and overlap matrices.

5.9 Visualization and Figure Construction

The main paper uses compact visual summaries rather than full heatmap grids for every configuration. Three types of figures are generated from saved artifacts:

1. loss summaries as a function of n and dtype, with min–median–max ranges over seeds;
2. near-exact pair count summaries as a function of n and dtype;
3. sorted pairwise-loss spectra for representative configurations.

Full pairwise heatmap grids are generated as supporting artifacts. These heatmaps visualize the matrices $|U_i^\dagger U_j|^2$ directly, but they become large as n increases. For example, the complete-target case $n = 7$ contains $\binom{7}{2} = 21$ pairwise overlap matrices. For this reason, the main text emphasizes compact quantitative summaries and pairwise-loss spectra. For each plotted figure, the numerical data used to generate the figure are saved separately from the rendered image, allowing figures to be regenerated without rerunning optimization.

5.10 Scope and Limitations of the Experiments

The experiments are numerical and do not constitute a proof of existence or nonexistence of complete MUB sets in dimension six. The optimizer is not guaranteed to find global minima, and the observed basins depend on initialization, precision, and optimizer settings.

The multi-seed campaigns reported here use 100 independent seeds per value of n for $n = 3, \ldots, 6$. This large-scale sweep provides a robust statistical characterization of the full nonconvex landscape and its basins.

The complex128 results are used as the primary reference because they provide stable near-exact classifications and small unitarity residuals. The complex64 results are interpreted as a precision-sensitivity study. Differences between complex128 and complex64 are therefore treated as evidence of precision-dependent optimization behavior rather than as a ranking of one precision as universally superior.

The reported landscape is also conditioned on the Lie-exponential parameterization and Adam optimizer; alternative manifold-optimization methods or parameterizations may sample different basins.

6 Results

This section reports the numerical results obtained from the unanchored AMUB workflow in dimension six. We first summarize the single-seed sweep over $n = 2, \ldots, 7$, then present the multi-seed complex128 campaign for $n = 3, \ldots, 6$, followed by the complex64 precision-sensitivity campaign. Finally, we interpret the pairwise defect spectra of representative configurations.

The results are numerical and should be interpreted as evidence about the optimization landscape explored by the workflow. They do not constitute a proof of existence or nonexistence of exact MUB configurations in dimension six.

6.1 Single-Seed Sweep over $n = 2, \ldots, 7$

As an initial validation and exploratory study, we ran the complex128 reference workflow for $n = 2, \ldots, 7$ using a fixed seed. The results are summarized in Table 2. This sweep verifies that the same parameterized workflow applies uniformly across the range from a single MUB pair to the complete-set target size $n = 7$.

For $n = 2$, the workflow recovered a mutually unbiased pair to numerical precision, with loss approximately 1.54×10^{-30}. This provides a positive control for the loss function and the Lie-exponential parameterization. For $n = 3$, the fixed-seed run reached a nonzero defective triangular configuration with total loss approximately 0.051249218996 and no near-exact pairs. For $n = 4$, the total loss was again approximately 0.051249218996, but the pairwise structure differed: three pairs were near-exact and three pairs

were defective. This equality of total loss between the fixed-seed $n = 3$ and $n = 4$ runs is explained by the pairwise decomposition: the $n = 4$ configuration contains a defective triangular component together with three near-exact hub edges.

For $n = 5$, the fixed-seed run produced a fully defective configuration with no near-exact pairs and total loss approximately 0.359638850581. The $n = 6$ fixed-seed run reached a recurrent nonzero basin near 0.965932502216, while the $n = 7$ fixed-seed run produced a fully defective complete-target configuration with loss approximately 1.584472133131. In both cases, every pairwise relation was defective under the primary near-exact tolerance.

The single-seed sweep suggests a progression from exact behavior at $n = 2$, through partially exact or structured defective configurations at $n = 3$ and $n = 4$, to fully defective configurations for $n \geq 5$. The following multi-seed campaigns test which of these features persist across random initialization.

6.2 Multi-Seed complex128 Campaign

The complex128 campaign used 100 independent seeds for each $n = 3, 4, 5, 6$. Table 3 summarizes the loss statistics and near-exact pair counts observed in the campaign.

For $n = 3$, the workflow reached both exact and defective basins. A majority of seeds produced configurations with three near-exact pairs and loss numerically indistinguishable from zero at the reported scale (with median loss $\sim 10^{-29}$). However, some seeds produced fully defective configurations, including the defective triangular basin near 0.051249218996 and another defective basin near 0.065592. Thus, even when exact triples are reachable by the workflow, the unanchored nonconvex landscape contains competing defective attractors.

For $n = 4$, a significant fraction of seeds converged to the same low-loss basin near 0.051249218996, with three near-exact pairs and three defective pairs. This is the hub-and-triangle structure identified in the single-seed analysis. The remaining seeds reached higher-loss fully defective basins. The repeated recovery of the 0.051249218996 basin suggests that the hub-and-triangle configuration is a prominent attractor in the complex128 workflow.

For $n = 5$, all 100 seeds produced fully defective configurations with no near-exact pairs. The best loss observed was approximately 0.359637, while the median and maximum best losses were approximately 0.439033 and 0.689537, respectively. Most importantly, no near-exact pair was observed in any of the 100 runs.

For $n = 6$, all 100 seeds were also fully defective. The campaign sampled multiple nonzero basins, including losses near 0.868532, 0.965933, and higher. The best observed complex128 loss in this campaign was 0.868532, and the median best loss was also 0.868532. These results strongly support the empirical evidence that the six-basis landscape sampled by the present unanchored workflow is fully defective under the primary near-exact tolerance.

The complex128 campaign supports a transition from exact or partially exact configurations for $n = 3$ and $n = 4$ to fully defective configurations for $n = 5$ and $n = 6$. In this sense, the observed transition begins at $n = 5$ in the present workflow.

6.3 Precision Sensitivity: complex128 versus complex64

We repeated the multi-seed campaign for $n = 3, 4, 5, 6$ using complex64 arithmetic with 100 independent random seeds. Table 4 compares the complex128 and complex64 summaries. As described in Section 5, these structural precision comparisons use the CPU matrix-exponential pathway rather than the Taylor GPU backend.

For $n = 3$, both precisions found exact and defective basins. In the complex64 campaign, a majority of seeds reached exact or near-exact triples, while others reached the defective triangular basin. Thus, the coexistence of exact and defective basins is observed in both precisions.

For $n = 4$, the complex64 campaign found low-loss configurations near 0.051249, but the near-exact classification was more sensitive to tolerance than in complex128. Under the primary tolerance 10^{-6}, the median near-exact count for complex64 was 0, while the maximum was 3. A subsequent tolerance check showed that some complex64 $n = 4$ runs recover three near-exact pairs only under relaxed tolerances such as 10^{-5} or 10^{-4}. By contrast, the complex128 $n = 4$ hub-and-triangle classifications were stable under the tolerances examined.

Table 2: Single-seed complex128 sweep over $n = 2, \ldots, 7$ in dimension six. Near-exact pairs are counted using the tolerance 10^{-6}.

n	Number of pairs	Best loss	Near-exact pairs	Defective pairs
2	1	1.543×10^{-30}	1	0
3	3	0.051249218996	0	3
4	6	0.051249218996	3	3
5	10	0.359638850581	0	10
6	15	0.965932502216	0	15
7	21	1.584472133131	0	21

Table 3: Multi-seed complex128 campaign for $n = 3, \ldots, 6$. The near-exact counts list the min/median/max number of near-exact pairs observed across 100 random seeds, using tolerance 10^{-6}.

n	Seeds	Loss min/median/max	Near-exact min/med/max
3	100	0.000000/0.000000/0.134080	0/3/3
4	100	0.051249/0.051249/0.294147	0/3/3
5	100	0.359637/0.439033/0.689537	0/0/0
6	100	0.868532/0.868532/1.404517	0/0/0

For $n = 5$, both precisions produced fully defective configurations in all 100 seeds. This is the strongest precision-stable transition result: no near-exact pairs were observed for $n = 5$ under either complex128 or complex64, even when relaxed near-exact tolerances were examined.

For $n = 6$, both precisions produced fully defective configurations. However, the observed basin sampling differed. Reduced precision can affect basin selection and attractor stability in the nonconvex AMUB landscape, while preserving the qualitative fully defective structure for $n = 6$.

The precision comparison suggests that the qualitative transition from partial exactness to fully defective configurations is stable across complex128 and complex64. At the same time, the detailed basin frequencies and some near-exact classifications are precision-sensitive.

6.4 Tolerance Sensitivity of Near-Exact Classification

Near-exact pair classification depends on a numerical threshold. The primary threshold used in the summary tables is 10^{-6}, but additional checks were performed at 10^{-5} and 10^{-4}.

For the complex128 campaign, the near-exact counts were unchanged across these tolerances. Exact or near-exact configurations remained classified as such, and fully defective configurations for $n = 5$ and $n = 6$ remained fully defective.

For the complex64 campaign, the main tolerance sensitivity occurred at $n = 4$. Some runs classified as having zero near-exact pairs under 10^{-6} exhibited three near-exact pairs under 10^{-5} or 10^{-4}. This indicates that reduced precision can affect the numerical classification of near-exact hub edges. However, the $n = 5$ and $n = 6$ complex64 runs remained fully defective under all examined tolerances. Therefore, within the examined tolerance range, the observed disappearance of near-exact pairs for $n \geq 5$ is not an artifact of the strict 10^{-6} threshold in the present experiments.

6.5 Pairwise Defect Spectra

The pairwise defect spectra provide a compact visualization of the structure hidden behind the scalar loss values. Each spectrum consists of the sorted pairwise losses ℓ_{ij} for a representative optimized configuration. The sorted pairwise-loss spectra for representative complex128 and complex64 configurations are shown in Appendix Figures 5 and 6, respectively.

The $n = 3$ exact representative has all pairwise losses at numerical zero. The $n = 3$ defective representative has three approximately equal nonzero pairwise losses, forming a symmetric defective triangle.

The $n = 4$ hub-and-triangle representative has

Table 4: Precision comparison for multi-seed AMUB campaigns in dimension six. Near-exact min/med/max use the primary tolerance 10^{-6} across 100 independent seeds.

n	Dtype	Seeds	Loss min/median/max	Near-exact min/med/max
3	complex128	100	0.000000/0.000000/0.134080	0/3/3
3	complex64	100	0.000000/0.000000/0.111767	0/3/3
4	complex128	100	0.051249/0.051249/0.294147	0/3/3
4	complex64	100	0.051249/0.051249/0.409379	0/0/3
5	complex128	100	0.359637/0.439033/0.689537	0/0/0
5	complex64	100	0.359636/0.439033/0.772279	0/0/0
6	complex128	100	0.868532/0.868532/1.404517	0/0/0
6	complex64	100	0.868532/0.868532/1.493699	0/0/0

three near-zero pairwise losses and three approximately equal defective losses. This supports the interpretation that the $n = 4$ low-loss basin consists of a defective three-basis core together with a near-exact hub basis.

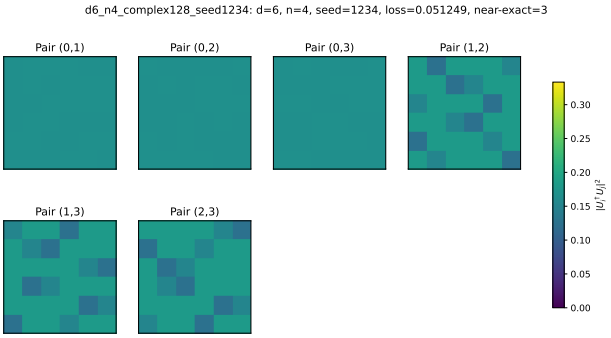

Figure 1: Representative pairwise-loss heatmap for $d = 6, n = 4$. The heatmap shows a hub-and-triangle structure with three near-zero losses and three nonzero losses.

For $n = 5$, all ten pairwise losses are nonzero in the representative best configuration. The spectrum is not uniform: one pair carries substantially more defect than the others in the selected representative. This indicates that the transition at $n = 5$ is not merely a uniform spreading of the $n = 4$ defect, but a reorganization into a fully defective configuration.

For $n = 6$, the representative low-loss basin has all fifteen pairwise losses nonzero, with repeated loss levels. This is consistent with a globally defective but structured configuration, in which no pair is near-exact under the primary tolerance. The sorted spectra therefore support the interpretation that increasing n changes the optimized configurations from exact or partially exact pairwise relations to globally distributed defect.

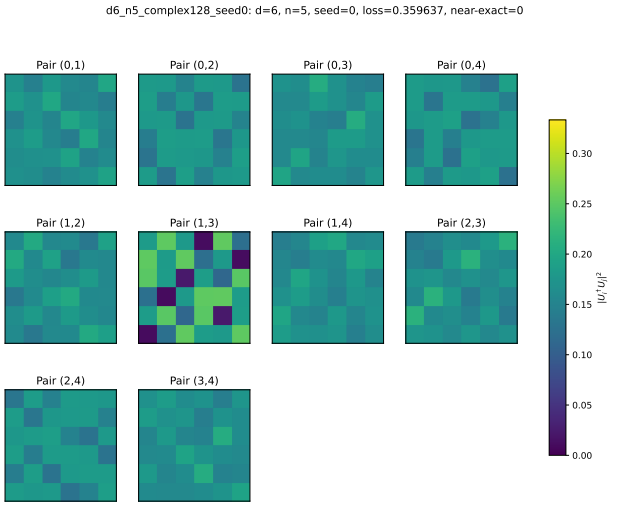

Figure 2: Representative pairwise-loss heatmap for $d = 6, n = 5$. The heatmap shows a fully defective structure with all pairs having nonzero loss.

6.6 Hardware Benchmarks and GPU Acceleration Performance

To evaluate the portable hardware-acceleration layer described in Section 4, we performed benchmark experiments on an Apple Silicon MacBook Pro (M1 processor) running PyTorch 2.4. We measured the average execution time (in seconds) required to perform 1000 optimization iterations for the case $n = 6$ bases, comparing CPU execution under reference `complex128` and baseline `complex64` precisions against the custom Taylor-series matrix exponential running natively on the Apple Silicon GPU via the `mps` backend. The results are summarized in Table 5.
Relative GPU speed is computed as CPU complex128 time divided by MPS GPU time; values above 1 indicate GPU speedup relative to the double-precision CPU baseline.

Table 5: Multi-dimensional hardware benchmark and iteration speed comparison (in seconds) per 1000 optimization steps ($n = 6$ bases) across dimensions $d = 6, 12, 24, 48, 96$ on CPU vs. Apple M1 GPU (MPS). Order-20 Taylor expansion is utilized for the GPU backend.

Dimension d	CPU `c128` (s)	CPU `c64` (s)	M1 GPU MPS (s)	Relative GPU Speed
6	2.617	2.695	42.958	0.06×
12	3.367	3.099	44.105	0.08×
24	4.547	4.032	43.345	0.10×
48	13.077	9.782	45.735	0.29×
96	62.281	37.244	48.384	1.29×

The benchmark results in Table 5 show a hardware crossover behavior typical of accelerator-backed dense linear algebra. For small dimensions, the Apple M1 GPU backend is substantially slower than the CPU baselines. At $d = 6$, the arithmetic workload is too small to amortize accelerator overheads, and the CPU completes 1000 iterations in under three seconds, while the MPS backend requires approximately 43 seconds.

As d increases, the CPU timings grow rapidly, reflecting the increasing cost of dense matrix products and matrix exponentiation. By contrast, the measured MPS timings remain in the range of approximately 43–48 seconds across the tested dimensions. This nearly constant GPU timing is consistent with fixed accelerator-launch overheads dominating the small- and medium-dimensional workloads.

At $d = 96$, the MPS Taylor backend completes 1000 iterations in 48.384 seconds, compared with 62.281 seconds for the CPU complex128 reference timing. Thus, the measured timings show a crossover between $d = 48$ and $d = 96$ relative to the double-precision CPU baseline. Interpolating these timings suggests a crossover near $d \approx 80$. The MPS backend remains slower than the CPU complex64 timing in this benchmark, so the observed speedup should be interpreted specifically relative to the complex128 CPU reference.

These results indicate that the Taylor-series layer enables portable accelerator execution and can become favorable at larger dimensions, while preserving a CPU-native reference pathway for the structural AMUB experiments reported above.

To evaluate the scalability of the approach on enterprise-grade hardware, we additionally performed benchmark experiments on the JANUS high-performance computing cluster, which features AMD EPYC 7352 CPUs and NVIDIA A100 GPUs. As shown in Table 6, the CPU baseline times grow significantly with dimension, while the NVIDIA A100 GPU maintains a nearly constant timing of approximately 26–28 seconds across all dimensions up to $d = 96$. The hardware crossover occurs earlier on the JANUS cluster, overtaking the double-precision CPU baseline between $d = 24$ and $d = 48$, and overtaking the single-precision CPU baseline at $d = 96$.

Comparing the two systems, the crossover occurs much sooner on the enterprise platform ($d \approx 30$) than on the consumer workstation ($d \approx 80$). This is explained by the A100's massive parallel processing capacity, which allows it to handle the larger matrix arithmetic workload of higher dimensions with negligible timing increase (growing by only 1.2 seconds from $d = 6$ to $d = 96$). While both devices exhibit similar flat execution curves dominated by initial API and kernel launch overheads (around 43–45 seconds on the M1 MPS and 26–28 seconds on the A100 CUDA), the significantly higher compute power of the A100 allows it to achieve a speedup relative to the CPU reference at a much lower dimensionality. This confirms that the Taylor-series matrix exponential fallback delivers substantial acceleration for moderate-to-large tensor dimensions on high-performance accelerators.

Relative GPU speed is computed as CPU complex128 time divided by GPU complex128 time.

7 Conclusion

This paper presented a generalized, reproducible mathematical software workflow for unanchored approximate MUB optimization. The workflow is dimension-agnostic, representing candidate bases by Lie-exponential unitary parameterizations, optimizing all bases simultaneously

Table 6: Multi-dimensional hardware benchmark and iteration speed comparison (in seconds) per 1000 optimization steps ($n = 6$ bases) across dimensions $d = 6, 12, 24, 48, 96$ on the JANUS HPC cluster (AMD EPYC 7352 CPU vs. NVIDIA A100 GPU). Order-20 Taylor expansion is utilized for the GPU backend.

Dimension d	CPU `c128` (s)	CPU `c64` (s)	A100 GPU (s)	Relative GPU Speed
6	7.270	7.649	26.495	0.27×
12	10.314	10.870	27.317	0.38×
24	22.282	15.492	27.453	0.81×
48	31.449	22.983	27.578	1.14×
96	72.810	55.595	27.705	2.63×

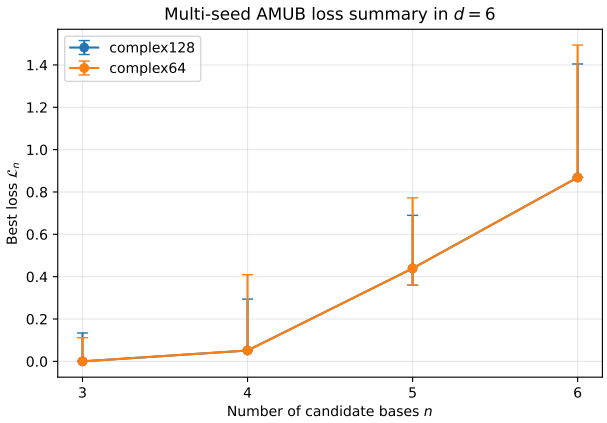

Figure 3: Best AMUB loss summary in $d = 6$. This plot shows the median best loss with min–max range across 100 seeds. The AMUB loss increases with n.

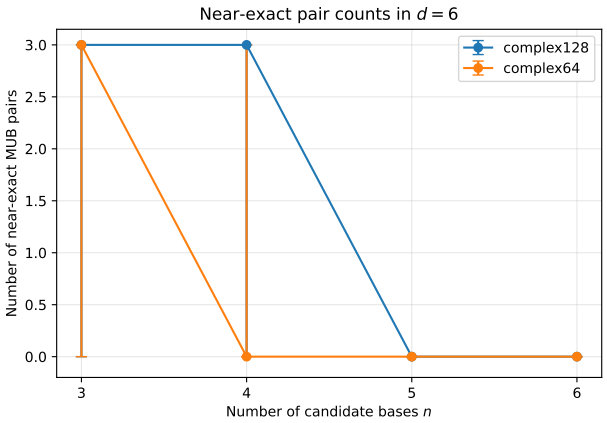

Figure 4: Near-exact MUB pair counts in $d = 6$. This plot shows the median number of near-exact MUB pairs with corresponding ranges across 100 seeds. Near-exact pairs disappear for $n \geq 5$ under the primary tolerance, coinciding with a transition to fully defective observed configurations.

without fixing a reference basis, and recording machine-readable artifacts. As a validation case study, we successfully applied the framework to dimension six, recovering exact configurations for small basis counts and demonstrating a robust structural transition to fully defective configurations for five or more bases.

The numerical experiments provide evidence, under the specified optimization protocol, for a transition in the observed unanchored AMUB landscape. Exact or partially exact configurations are recovered for smaller numbers of bases, including exact triples and four-basis hub-and-triangle configurations. In contrast, the reported campaigns for $n = 5$ and $n = 6$ produce fully defective configurations under the examined near-exact tolerances. Pairwise defect spectra show that this transition is not only a change in scalar loss, but also a change in the structure of the residual pairwise defects.

The precision comparison shows that the qualitative transition to fully defective configurations for $n \geq 5$ is stable across complex128 and complex64 in the reported campaigns, while basin selection and some near-exact classifications remain precision-sensitive. The hardware benchmarks further show that the Taylor-series matrix-exponential layer enables portable accelerator execution and becomes favorable relative to the CPU complex128 reference at larger tested dimensions.

The results are numerical and do not constitute a proof of existence or nonexistence of complete MUB sets in dimension six. They are conditioned on the Lie-exponential parameterization, Adam optimizer, selected tolerances, and finite seed campaigns. To support reproducibility and community extensions, the complete software codebase and all multi-seed results are hosted openly on GitHub at `https://github.com/fatahjamro/d6mubOptimization-v2`. Future work includes larger seed sweeps, alternative manifold-optimization methods, extended preci-

sion studies, and broader searches in other composite dimensions.

Acknowledgments

This work was supported by Atlantic Technological University, Ireland, under the Postgraduate Research Training Programme in Modelling and Computation for Health and Society (MOCHAS-PRTP). We also acknowledge the ATU JANUS High-Performance Computing (HPC) cluster facility for providing the computational resources required for the multi-seed campaigns.

References

[1] Andreas Klappenecker and Martin Rötteler. "Constructions of mutually unbiased bases". In Finite Fields and Applications: 7th International Conference, Fq7, Toulouse, France, May 5-9, 2003. Revised Papers. Pages 137–144. Springer (2004).

[2] Charles H. Bennett and Gilles Brassard. "Quantum cryptography: Public key distribution and coin tossing". Theoretical Computer Science **560**, 7–11 (2014).

[3] William K Wootters and Brian D Fields. "Optimal state-determination by mutually unbiased measurements". Annals of Physics **191**, 363–381 (1989).

[4] Daniel McNulty and Stefan Weigert. "Mutually Unbiased Bases in Composite Dimensions - A Review". Quantum **10**, 2051 (2026).

[5] Ingemar Bengtsson, Wojciech Bruzda, Åsa Ericsson, Jan-Åke Larsson, Wojciech Tadej, and Karol Życzkowski. "Mutually unbiased bases and hadamard matrices of order six". Journal of Mathematical Physics **48**, 052106 (2007).

[6] Paweł Horodecki, Łukasz Rudnicki, and Karol Życzkowski. "Five open problems in quantum information theory". PRX Quantum **3**, 010101 (2022).

[7] Stephen Brierley, Stefan Weigert, and Ingemar Bengtsson. "All mutually unbiased bases in dimensions two to five". Quantum Information & Computation **10**, 803–820 (2010).

[8] Elena Celledoni and Arieh Iserles. "Approximating the exponential from a lie algebra to a lie group". Math. Comput. **69**, 1457–1480 (2000).

[9] Gilbert Helmberg. "Introduction to spectral theory in hilbert space". Courier Dover Publications. (2008). url: https://www.sciencedirect.com/science/book/9780720423563.

[10] Dudley Ernest Littlewood. "The theory of group characters and matrix representations of groups". Volume 357. American Mathematical Soc. (1977).

[11] THOMAS DURT, BERTHOLD-GEORG ENGLERT, INGEMAR BENGTSSON, and KAROL ŻYCZKOWSKI. "On mutually unbiased bases". International Journal of Quantum Information **08**, 535–640 (2010).

[12] Stephen Brierley. "Mutually unbiased bases in low dimensions". PhD thesis. University of York. (2009). url: https://etheses.whiterose.ac.uk/id/eprint/587/.

[13] Markus Grassl. "On SIC-POVMs and MUBs in Dimension 6" (2004). arXiv:quant-ph/0406175.

[14] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. "Automatic differentiation in pytorch". In NIPS 2017 Workshop on Autodiff. (2017). url: https://openreview.net/forum?id=BJJsrmfCZ.

[15] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. "Pytorch: An imperative style, high-performance deep learning library". In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, Advances in Neural Information Processing Systems 32. Pages 8024–8035. Curran Associates, Inc. (2019).

[16] Nicholas J. Higham. "Functions of matrices". Society for Industrial and Applied Mathematics. (2008).

[17] B. J. Musto. "Quantum latin squares and quantum functions: Applications in quantum information". PhD thesis. University of Oxford. (2019).

url: https://www.cs.ox.ac.uk/people/bob.coecke/BenMustoThesis.

[18] Emanuel Knill. "Non-binary unitary error bases and quantum codes" (1996). arXiv:quant-ph/9608048.

[19] David J. Reutter and Jamie Vicary. "Biunitary constructions in quantum information". Higher Structures **3**, 109–154 (2019).

8 Appendix: Additional Figures

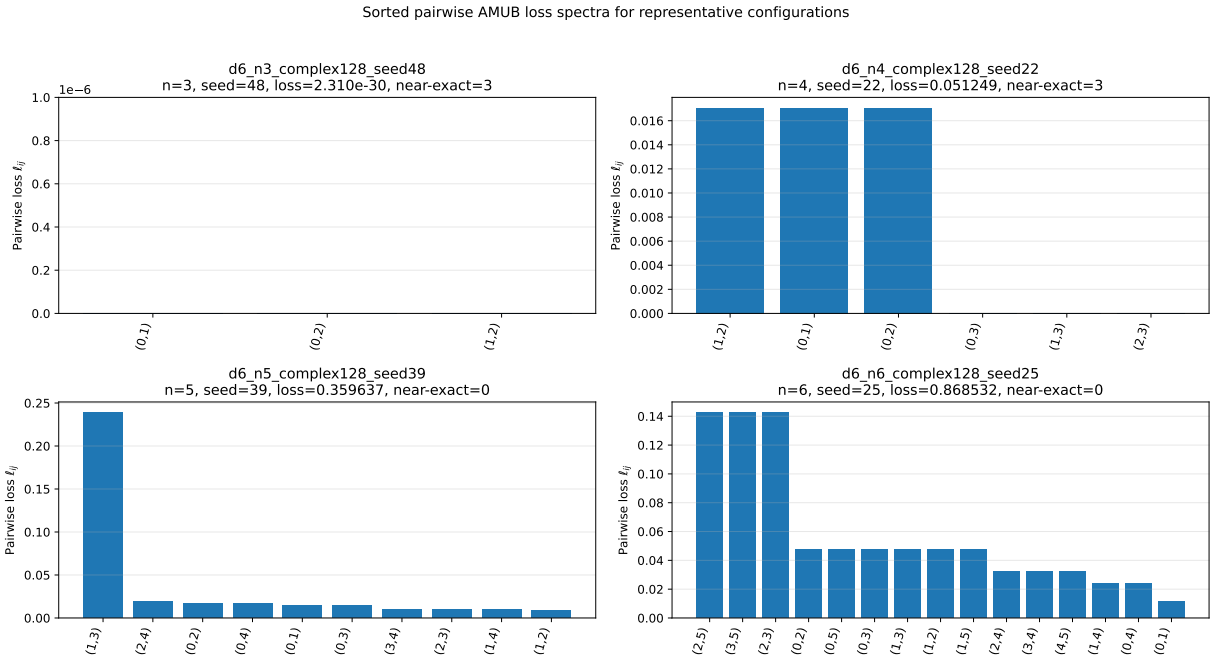

Figure 5: Sorted pairwise AMUB loss spectra for representative complex128 configurations. Each bar corresponds to one pairwise contribution ℓ_{ij}. The exact $n = 3$ representative has pairwise losses at numerical zero, the defective $n = 3$ representative has three approximately equal nonzero losses, the $n = 4$ representative exhibits three near-zero losses and three equal defective losses, and the $n = 5$ and $n = 6$ representatives have no zero pairwise losses.

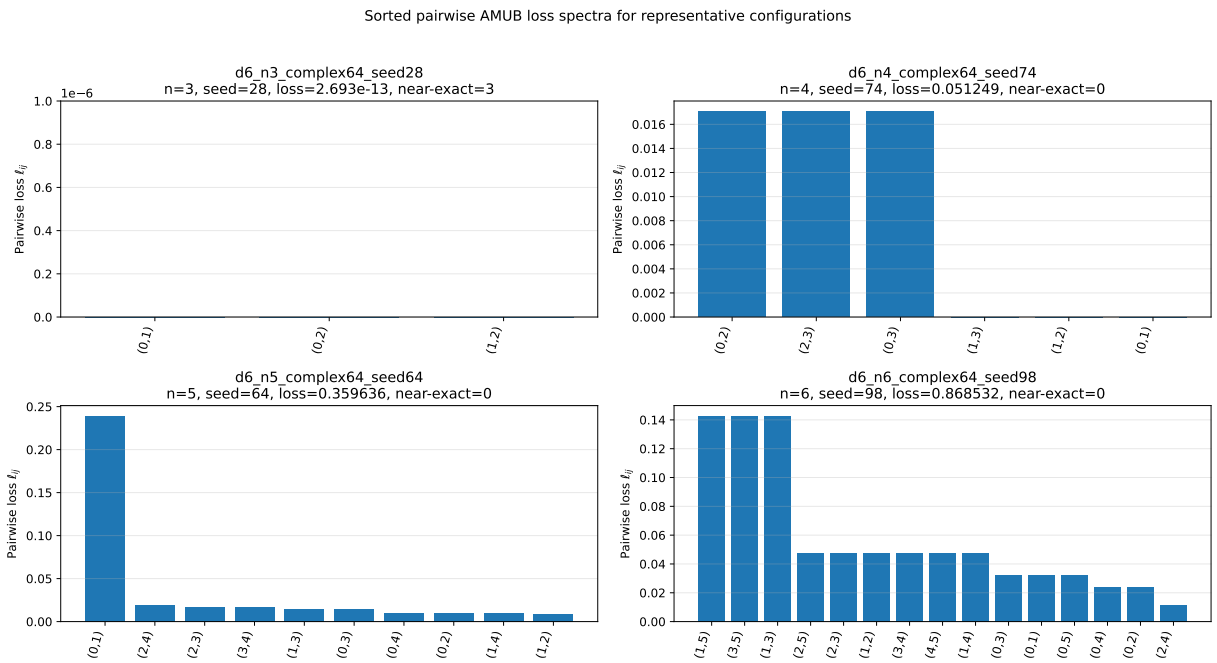

Figure 6: Sorted pairwise AMUB loss spectra for representative complex64 configurations. Machine-zero losses below the display tolerance are shown as zero for visualization only; raw values are retained in the saved figure-data artifact. The figure illustrates that the main transition to fully defective configurations for $n = 5$ and $n = 6$ persists in complex64, while the sampled basins and near-exact classifications differ from the complex128 reference campaign.